Exercise 1

Sound Classification Input Sketch

Music and Multimedia Streaming Lab
README — Processing + Wekinator

Current Successful Configuration

Feature mode	3 — 12 reduced FFT bands
Wekinator inputs	12
Wekinator outputs	1
Output type	Classification
Classifier	K-Nearest Neighbours, $k = 3$
Class 1	`eeeeeeeeeeeee`
Class 2	`uuuuuuuuuuuuu`

Contents

1 Purpose of This Report **1**

2 Big Picture of the Project **1**

3 What This Processing Sketch Is **2**

4 Libraries Used **2**
4.1 `processing.sound.*` 2
4.2 `oscP5.*` 2
4.3 `netP5.*` 2

5 Complete Code **2**

6 Block-by-Block Code Explanation **6**
6.1 Audio Object Declarations 6
6.2 Feature Mode Selection 6
6.3 Smoothing 6
6.4 OSC Ports 7

7 The `setup()` Function **7**

8 The `analyzeAudio()` Function **7**
8.0.1 Mode 1 — Full 512-band FFT 7
8.0.2 Mode 2 & 3 — Reduced FFT via downsampling 7
8.0.3 Mode 4 — Mean and Variance 8

9 The `sendFeaturesToWekinator()` Function **8**

10 Feature Dimensionality Explained **8**

11 FFT Explained **9**

12 Lab Requirements Checklist **9**

13 Wekinator Configuration Table **10**

14 Completed Experiment Details **10**

15 How to Run the Complete System **11**

16 Why One Output for Two Classes? **11**

17 Expected Performance of Each Feature Mode **11**
17.1 Mode 1 — 512 FFT Bands 11
17.2 Mode 2 — 64 FFT Bands 11
17.3 Mode 3 — 12 FFT Bands (used in our experiment) 11
17.4 Mode 4 — Mean + Variance 12

18 Why RMS is Not Included **12**

19 Debugging Checklist **12**
19.1 Wekinator Does Not Receive Data 12
19.2 Classification Does Not Work Well 12

19.3 Output Sketch Does Not Change Colour 12

20 Summary **13**

1. Purpose of This Report

This README explains the Processing input sketch used for **Exercise 1 — Sound Classification** in the Music and Multimedia Streaming lab.

The goal of the sketch is to capture live microphone audio, extract numerical audio features, and send those features to Wekinator using OSC. Wekinator then uses those features to train and run a sound classifier.

This report explains:

1. What the sketch does in the project.
2. How the Processing code works as a standalone program.
3. Each important line or group of lines in the code.
4. How FFT, downsampling, smoothing, mean, and variance are used.
5. How this sketch fulfils the first exercise requirements.
6. How to use this sketch with Wekinator.
7. How to document the experiment in the final lab report.

2. Big Picture of the Project

The first exercise asks us to build a real-time sound classification system. The system has three main parts:

```
Microphone
↓
Processing input sketch
↓
Audio feature extraction
↓
OSC message /wek/inputs
↓
Wekinator classifier
↓
OSC message /wek/outputs
↓
Processing output sketch
↓
Color changes according to the predicted class
```

This README focuses on the **input sketch**, which is responsible for:

- Listening to live microphone sound.
- Computing an FFT with 512 frequency bands.
- Creating different feature representations: 512, 64, and 12 reduced FFT bands, plus mean and variance.
- Sending the selected feature vector to Wekinator via OSC.
- Showing a real-time visualization of the extracted audio features.

Feature mode: 3 **Representation:** 12 reduced FFT bands
Classifier: KNN, $k = 3$ **Classes:** 2
Class 1 = `eeeeeeeeeeeee` Class 2 = `uuuuuuuuuuuuu`

3. What This Processing Sketch Is

This Processing file is an **input sketch**, not the visualization output sketch.

This sketch does **not** receive Wekinator predictions and does **not** change the screen color based on the class. That is the job of the second Processing sketch.

This sketch handles the first half of the system:

```
Live microphone audio → numerical features → Wekinator
```

The sketch sends data to Wekinator using:

- OSC address: `/wek/inputs`
- Port: `6448` (standard Wekinator input port)

4. Libraries Used

```
import processing.sound.*;
import oscP5.*;
import netP5.*;
```

Listing 1: Library imports

4.1 `processing.sound.*`

Gives Processing access to audio input and analysis tools. Used for `AudioIn` (live microphone) and `FFT` (frequency analysis).

4.2 `oscP5.*`

Allows Processing to send and receive OSC messages.
OSC = Open Sound Control — the communication protocol between Processing and Wekinator.

4.3 `netP5.*`

Provides network address tools used together with `oscP5`. Used to define the destination `127.0.0.1:6448` (localhost, same computer).

5. Complete Code

```
import processing.sound.*;
import oscP5.*;
import netP5.*;

```

```
// ----------------------------
// AUDIO
// ----------------------------
AudioIn mic;
FFT fft;

int bands = 512;
float[] spectrum = new float[bands];

// ----------------------------
// FEATURE EXTRACTION
// ----------------------------
// 1 = full FFT, 512 inputs
// 2 = reduced FFT, 64 inputs
// 3 = reduced FFT, 12 inputs
// 4 = mean + variance, 2 inputs
int featureMode = 3;

int numFeatureBands;
float[] features;
float[] smoothFeatures;

float smoothing = 0.85;

// ----------------------------
// OSC / WEKINATOR
// ----------------------------
OscP5 oscP5;
NetAddress wekinator;

int receivePort = 12001;
int sendPort    = 6448;

int sendIntervalMs = 30;
int lastSendTime   = 0;

void setup() {
  size(1000, 600);
  surface.setTitle("Realtime Sound Classifier - Processing + Wekinator");

  oscP5      = new OscP5(this, receivePort);
  wekinator  = new NetAddress("127.0.0.1", sendPort);

  mic = new AudioIn(this, 0);
  mic.start();

  fft = new FFT(this, bands);
  fft.input(mic);

  configureFeatureMode();
  textFont(createFont("Arial", 16));
}

void configureFeatureMode() {
  if      (featureMode == 1) numFeatureBands = 512;
  else if (featureMode == 2) numFeatureBands = 64;
  else if (featureMode == 3) numFeatureBands = 12;
```

```
  else if (featureMode == 4) numFeatureBands = 2;

  features       = new float[numFeatureBands];
  smoothFeatures = new float[numFeatureBands];
}

void draw() {
  background(15);
  analyzeAudio();

  if (millis() - lastSendTime >= sendIntervalMs) {
    sendFeaturesToWekinator();
    lastSendTime = millis();
  }

  drawUI();
}

void analyzeAudio() {
  fft.analyze(spectrum);

  if (featureMode == 1) {
    for (int i = 0; i < bands; i++) {
      smoothFeatures[i] = lerp(smoothFeatures[i], spectrum[i], 1.0
          - smoothing);
      features[i] = smoothFeatures[i] * 10.0;
    }
  } else if (featureMode == 2) {
    downsampleSpectrum(64);
  } else if (featureMode == 3) {
    downsampleSpectrum(12);
  } else if (featureMode == 4) {
    float mean     = computeMean(spectrum);
    float variance = computeVariance(spectrum, mean);
    features[0] = mean;
    features[1] = variance;
  }
}

void downsampleSpectrum(int targetBands) {
  for (int i = 0; i < targetBands; i++) {
    int start = floor(i * bands / float(targetBands));
    int end   = floor((i + 1) * bands / float(targetBands));

    float sum = 0;
    for (int j = start; j < end; j++) sum += spectrum[j];

    float avg = sum / float(end - start);
    smoothFeatures[i] = lerp(smoothFeatures[i], avg, 1.0 -
        smoothing);
    features[i] = smoothFeatures[i] * 10.0;
  }
}

float computeMean(float[] values) {
  float sum = 0;
  for (int i = 0; i < values.length; i++) sum += values[i];
  return sum / values.length;
```

```
}

float computeVariance(float[] values, float mean) {
  float sum = 0;
  for (int i = 0; i < values.length; i++) {
    float d = values[i] - mean;
    sum += d * d;
  }
  return sum / values.length;
}

void sendFeaturesToWekinator() {
  OscMessage msg = new OscMessage("/wek/inputs");
  for (int i = 0; i < features.length; i++) msg.add(features[i]);
  oscP5.send(msg, wekinator);
}

void drawUI() {
  fill(255);
  textSize(24);
  text("Real time sound classification", 30, 40);

  textSize(18);
  text("Feature mode: " + featureMode, 30, 80);
  text("Inputs sent to Wekinator: " + features.length, 30, 110);

  if      (featureMode == 1) text("Mode 1: Full FFT with 512 bands",    30, 140);
  else if (featureMode == 2) text("Mode 2: Reduced FFT with 64 bands",  30, 140);
  else if (featureMode == 3) text("Mode 3: Reduced FFT with 12 bands",  30, 140);
  else if (featureMode == 4) text("Mode 4: Mean + variance",
                                 30, 140);

  int displayBands = min(numFeatureBands, 64);
  int x0 = 30, y0 = 200, gap = 3, maxH = 190;
  int barW = max(2, int((width - 60 - gap*(displayBands-1)) / float(displayBands)));

  for (int i = 0; i < displayBands; i++) {
    float v = constrain(features[i], 0, 1.5);
    float h = map(v, 0, 1.5, 0, maxH);
    fill(80, 180, 255);
    rect(x0 + i*(barW+gap), y0+maxH-h, barW, h);
  }

  stroke(0, 255, 140);
  noFill();
  beginShape();
  for (int i = 0; i < bands; i++) {
    float x = map(i, 0, bands-1, 30, width-30);
    float y = map(spectrum[i], 0, 0.2, height-30, height-160);
    vertex(x, y);
  }
  endShape();

  noStroke();
```

```
  fill(180);
  textAlign(LEFT);
  textSize(14);
  text("Send OSC -> /wek/inputs", 30, height-90);
  text("Press T to print feature values", 30, height-70);
}

void keyPressed() {
  if (key == 't' || key == 'T') {
    print("FEATURES: ");
    for (int i = 0; i < features.length; i++)
      print(nf(features[i], 1, 4) + " ");
    println();
  }
}
```

Listing 2: Full Processing input sketch

6. Block-by-Block Code Explanation

6.1 Audio Object Declarations

```
AudioIn mic;    // microphone input object
FFT fft;        // frequency analyser
```

These lines *declare* the variables. The microphone has not started yet — the lines only create the variable names.

```
int bands = 512;
float[] spectrum = new float[bands];
```

`bands = 512` matches the lab requirement. After every `fft.analyze(spectrum)` call, `spectrum[0]` to `spectrum[511]` contain the current frequency magnitudes: low frequencies on the left, high frequencies on the right.

6.2 Feature Mode Selection

```
int featureMode = 3;
```

`featureMode`	Meaning	Wekinator inputs
1	Full FFT	512
2	Reduced FFT	64
3	Reduced FFT	12
4	Mean + variance	2

6.3 Smoothing

```
float smoothing = 0.85;
// Used as:
smoothFeatures[i] = lerp(smoothFeatures[i], spectrum[i], 1.0 -
    smoothing);
```

Since `1.0 - 0.85 = 0.15`, the smoothed feature moves only 15% toward the new raw value per frame.

High smoothing → slower, more stable changes. Low smoothing → faster, more jumpy changes.
`0.85` is a good default for real-time audio classification.

6.4 OSC Ports

```
int receivePort = 12001;  // this sketch listens here (avoid conflict with output sketch)
int sendPort    = 6448;   // Wekinator default input port -- must match!
int sendIntervalMs = 30;  // ~33 messages per second
```

`sendPort = 6448` must equal the *Input Port* in Wekinator. Do not change it unless you also change Wekinator.

7. The `setup()` Function

```
void setup() {
  size(1000, 600);
  oscP5     = new OscP5(this, receivePort);
  wekinator = new NetAddress("127.0.0.1", sendPort);
  mic = new AudioIn(this, 0);
  mic.start();
  fft = new FFT(this, bands);
  fft.input(mic);
  configureFeatureMode();
}
```

Key connections established inside `setup()`:

`mic (microphone)` $\xrightarrow{\texttt{fft.input(mic)}}$ `FFT analyser` $\xrightarrow{\texttt{OSC}}$ `Wekinator @ 127.0.0.1:6448`

8. The `analyzeAudio()` Function

`fft.analyze(spectrum)` converts the live microphone signal into 512 frequency-band magnitudes every frame.

8.0.1 Mode 1 — Full 512-band FFT

All 512 values are smoothed and multiplied by 10 before sending.

8.0.2 Mode 2 & 3 — Reduced FFT via downsampling

```
void downsampleSpectrum(int targetBands) {
  for (int i = 0; i < targetBands; i++) {
    int start = floor(i * bands / float(targetBands));
    int end   = floor((i+1) * bands / float(targetBands));
```

```

    float sum = 0;
    for (int j = start; j < end; j++) sum += spectrum[j];

    float avg = sum / float(end - start);
    smoothFeatures[i] = lerp(smoothFeatures[i], avg, 1.0 -
        smoothing);
    features[i] = smoothFeatures[i] * 10.0;
  }
}
```

The proportional index calculation ensures all 512 bins are covered even when 512 is not evenly divisible by the target (e.g. 512/12 has a remainder of 8).

8.0.3 Mode 4 — Mean and Variance

```
float mean     = computeMean(spectrum);
float variance = computeVariance(spectrum, mean);
features[0] = mean;
features[1] = variance;
```

Statistic	Audio meaning
Mean	Average spectral energy across all 512 bands — describes overall loudness/brightness.
Variance	Spread of energy around the mean — high variance means the spectrum has strong peaks; low variance means energy is uniform.

9. The `sendFeaturesToWekinator()` Function

```
void sendFeaturesToWekinator() {
  OscMessage msg = new OscMessage("/wek/inputs");
  for (int i = 0; i < features.length; i++) msg.add(features[i]);
  oscP5.send(msg, wekinator);
}
```

A conceptual OSC packet in mode 3 looks like:

```
/wek/inputs  0.03  0.10  0.08  0.05  0.02  0.01  0.00  0.00  0.00
    0.00  0.00  0.00
```

Wekinator receives these 12 floats as its input feature vector.

10. Feature Dimensionality Explained

Mode	Type	# inputs	Description
1	Full FFT	512	Very detailed spectrum — all frequency bins
2	Reduced FFT	64	Medium detail — good balance
3	Reduced FFT	12	Low detail — fast training, our working mode
4	Mean + variance	2	Very compact — may lose spectral shape

High-dimensional features contain more information but may require more training examples and can be noisier. Low-dimensional features are simpler and train faster, at the cost of finer detail.

11. FFT Explained

FFT (Fast Fourier Transform) converts a short time-domain audio window into frequency-domain information.

```
Time-domain (waveform) ──FFT──> 512 frequency magnitudes
```

Different sounds tend to have different spectral patterns:

- Vowel `eeee` → energy concentrated in specific formant regions.
- Vowel `uuuu` → different formant distribution.
- Clap → broadband burst across many frequencies.

Wekinator does not understand sound directly — it understands *numbers*. FFT converts sound into the numbers Wekinator needs.

12. Lab Requirements Checklist

Lab requirement	Where fulfilled
Capture live microphone audio	`AudioIn` + `mic.start()`
Compute FFT	`fft.analyze(spectrum)`
Use 512 FFT bands	`int bands = 512`
Create 64-band version	`downsampleSpectrum(64)`
Create 12-band version	`downsampleSpectrum(12)`
Compute mean	`computeMean(spectrum)`
Compute variance	`computeVariance(spectrum, mean)`
Send features via OSC	`sendFeaturesToWekinator()`
OSC address `/wek/inputs`	`new OscMessage("/wek/inputs")`
Wekinator port 6448	`int sendPort = 6448`
Support four experiments	`featureMode = 1, 2, 3, or 4`

13. Wekinator Configuration Table

`featureMode`	Wekinator # inputs	Description
1	512	Full FFT
2	64	Reduced FFT
3	12	Reduced FFT
4	2	Mean + variance

The number of Wekinator inputs **must** equal `features.length` in Processing. A mismatch causes silent errors — Wekinator will receive the wrong number of values.

For all experiments, use the following Wekinator output settings:

```
# outputs:        1
Output type:      classification
# classes:        2
Classifier:       K-Nearest Neighbor
k:                3
Output address:   /wek/outputs
Output port:      12000
```

14. Completed Experiment Details

Completed Experiment — featureMode = 3

Input OSC address	`/wek/inputs`
Input port	`6448`
Number of inputs	`12`
Output OSC address	`/wek/outputs`
Output port	`12000`
Number of outputs	`1`
Output type	classification
Number of classes	2
Model type	K-Nearest Neighbor, $k = 3$
Class 1 sound	`eeeeeeeeeeeee`
Class 2 sound	`uuuuuuuuuuuuu`
Total examples	1081
Class 1 examples	560
Class 2 examples	521

The dataset is **balanced**: 560 vs 521 examples is close enough that the classifier is not biased toward one class.

15. How to Run the Complete System

1. **Start Wekinator** — set # inputs = 12, # outputs = 1, type = classifier, 2 classes.
2. **Run the input Processing sketch** — confirm the screen shows:
   `Feature mode: 3 Inputs sent to Wekinator: 12`
3. **Click Start Listening** in Wekinator — the OSC indicator should show incoming data.
4. **Record training examples**:
   - Select class 1, say `eeeeeeeeeeeee`
   - Select class 2, say `uuuuuuuuuuuuu`
5. **Click Train** in Wekinator.
6. **Click Run** in Wekinator.
7. **Run the output Processing sketch** — it listens on port 12000 for `/wek/outputs` and changes canvas colour according to the predicted class.

While **recording examples**, only the input sketch and Wekinator need to run.
During **final testing**, all three parts must run simultaneously:
Input sketch + Wekinator (Run mode) + Output sketch

16. Why One Output for Two Classes?

Wekinator uses `# outputs = 1` with `number of classes = 2`. The single output value encodes the predicted class:

Output value	Meaning
1	Class 1 (`eeeee`)
2	Class 2 (`uuuuu`)

One output is all that is needed to represent a multi-class label in Wekinator's classification mode.

17. Expected Performance of Each Feature Mode

17.1 Mode 1 — 512 FFT Bands

Advantages: Full spectral detail; distinguishes subtle differences.
Disadvantages: High dimensionality; may need many examples; can be noisy.

17.2 Mode 2 — 64 FFT Bands

Advantages: Good balance between detail and simplicity.
Disadvantages: Loses fine-grained spectral detail.

17.3 Mode 3 — 12 FFT Bands (used in our experiment)

Advantages: Easy and fast to train; stable; less likely to overfit.
Disadvantages: Loses fine frequency information; may confuse similar-sounding classes.

17.4 Mode 4 — Mean + Variance

Advantages: Very simple; lowest computational cost.
Disadvantages: Loses almost all spectral shape. Vowels like `eeee` and `uuuu` differ mainly in spectral shape, so this mode is the weakest for this task.

Spectral features (modes 1–3) describe how energy is distributed across frequencies — highly informative for vowel classification.
Statistical features (mode 4) compress the whole spectrum into two numbers — compact but usually less discriminative.

18. Why RMS is Not Included

An earlier version of the sketch included RMS (loudness) as an additional input. This was removed because the lab requires *exactly* four configurations:

512 bands / 64 bands / 12 bands / mean + variance

Adding RMS would change the input count (e.g. mode 3 would send 13 inputs instead of 12), breaking the Wekinator configuration.

19. Debugging Checklist

19.1 Wekinator Does Not Receive Data

- Input sketch is running.
- Wekinator is listening on port `6448`.
- Input OSC address is `/wek/inputs`.
- # inputs in Wekinator equals `features.length`.
- No library errors in the Processing console.
- Microphone is working and not muted.

19.2 Classification Does Not Work Well

- Record more examples for each class.
- Keep consistent microphone distance.
- Make class sounds clearly distinct.
- Avoid background noise.
- Confirm KNN uses $k = 3$.
- Re-train after changing model settings.

19.3 Output Sketch Does Not Change Colour

- Wekinator is in **Run** mode (not Train).
- Output sketch is running.

- Wekinator sends to port `12000` at address `/wek/outputs`.
- Output sketch listens on port `12000`.

20. Summary

This Processing sketch successfully implements the input side of the sound classification system. Changing a single variable:

```
int featureMode = 3;  // change to 1, 2, or 4 for other experiments
```

allows the same code to be reused across all four required experiments. The current tested configuration (`featureMode = 3`) correctly sends 12 FFT features to Wekinator and has been successfully trained for two vocal classes: `eeeeee` and `uuuuuu`.

Key Features of this Sketch

- Real-time microphone input via `AudioIn`
- 512-band FFT analysis via `FFT`
- Configurable feature mode (1, 2, 3, 4)
- Correct OSC communication with Wekinator on port 6448
- Real-time bar-graph and spectrum visualization
- Press **T** to print the feature vector to the console